

Towards context-aware graph representation learning: Adaptive node aggregation with LLMs

Songwei Zhao ^{a,b}, Yuan Jiang ^{c,*}, Sinuo Zhang ^{a,b}, Jifeng Hu ^{a,b},
Philip S. Yu ^d, Hechang Chen ^{a,b,*}

^a School of Artificial Intelligence, Jilin University, Changchun, China

^b Engineering Research Center of Knowledge-Driven Human-Machine Intelligence, Ministry of Education, China

^c School of Computer Science and Engineering, Nanyang Technological University, Singapore

^d Department of Computer Science, University of Illinois Chicago, Chicago, United States

ARTICLE INFO

Keywords:

Text-attributed graph
Graph neural network
Large language model
Context-aware aggregation

ABSTRACT

Representation learning on graph-structured data is a crucial research area in graph machine learning, with widespread application in real-world domains such as social networks and recommender systems. Traditional graph neural networks (GNNs) typically rely on static graph aggregation functions (e.g., mean, sum) applied over shallow textual features, which limits their ability to model complex semantic interactions among nodes. Recent advances have introduced large language models (LLMs) to enhance node features through deep semantic encoding. However, most existing work treats LLMs as static encoders, overlooking their potential to reason about inter-node relationships. In this work, we propose **LLM-based Tailored Node Aggregation (LTNA)**, a novel approach that leverages the in-context reasoning capabilities of LLMs to generate adaptive, semantics-driven aggregation strategies for each node. Rather than relying on a fixed rule or global function, LTNA prompts the LLM to infer the relational meaning between a node and its neighbors (e.g., supportive, critical, background) and outputs a customized aggregation strategy based on the inferred semantics. This enables message passing in GNNs to move beyond numeric aggregation and become a context-sensitive semantic operation. By jointly leveraging node textual attributes and graph structure, LTNA enhances the expressiveness of node representations and improves performance across downstream tasks. Extensive experiments on five benchmark datasets demonstrate that LTNA consistently outperforms state-of-the-art GNN models, achieving an average improvement of 7.95% in classification accuracy and 9.90% in Micro-F1 score.

1. Introduction

Graph-structured web data are prevalent in many real-world scenarios, such as user relationships in social networks or web links on the internet, where these graphs represent objects and their interrelationships through nodes and edges [1–3]. In many graphs, nodes contain not only structured connectivity information but are also associated with rich textual attributes, forming text-attributed graphs (TAGs) [4–6]. For example, in social networks, each user node may include textual descriptions such as personal profiles, while in citation networks, paper nodes are often accompanied by title and abstract. In TAGs, both the graph topology and the textual

* Corresponding authors.

E-mail addresses: zhaosw22@mails.jlu.edu.cn (S. Zhao), yuan005@ntu.edu.sg (Y. Jiang), snzhang22@mails.jlu.edu.cn (S. Zhang), hujf21@mails.jlu.edu.cn (J. Hu), psyu@uic.edu (P.S. Yu), chenhc@jlu.edu.cn, yuan005@ntu.edu.sg (H. Chen).

<https://doi.org/10.1016/j.artint.2026.104584>

Received 23 June 2025; Received in revised form 23 April 2026; Accepted 6 June 2026

Available online 14 June 2026

0004-3702/© 2026 Elsevier B.V. All rights are reserved, including those for text and data mining, AI training, and similar technologies.

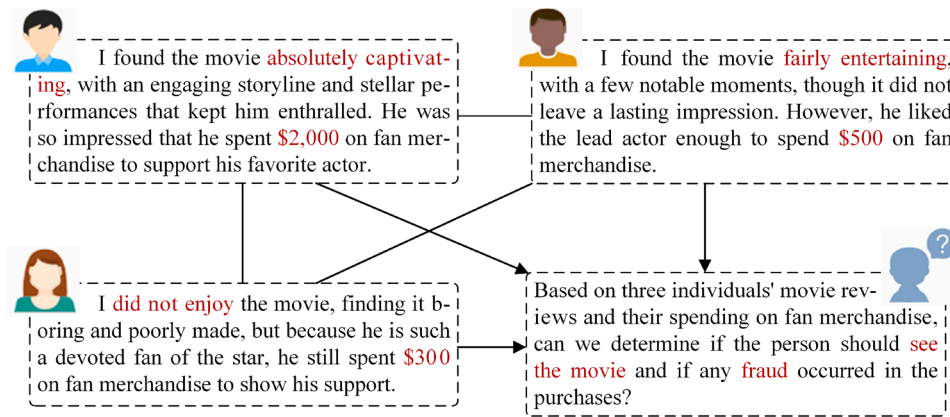


Fig. 1. Illustration of different aggregation methods in social graph. In the movie recommendation, mean aggregation captures the user's general preferences, while max or min aggregation could skew results toward extreme ratings. In purchasing fan merchandise transactions, mean aggregation dilutes the impact of an anomalously large transaction, whereas max aggregation preserves the anomaly, aiding in the detection of potential risks.

attributes of the nodes serve as key sources of information. This combination provides rich data support for representation learning and significantly enhances the performance of essential tasks such as node classification, link prediction, and graph classification.

In previous research, node representations learning of TAGs have typically processed textual attributes through simple text encoding techniques combined with aggregation operations in traditional graph neural networks (GNNs). Conventional pipelines to text encoding use shallow embedding methods such as Word2Vec [7] and Bag-of-Words [8], which transform textual information into a fixed vector representation. However, recent studies have shown that these non-contextualized shallow embeddings have certain limitations, such as difficulty in capturing the contextual information from polysemous words [9] and deficiency in understanding the deep text semantics [10]. In contrast to these shallow embeddings, Large Language Models (LLMs) exhibit strong context awareness and superior semantic understanding through pre-training on large-scale text corpora [11–13]. LLMs are capable of generating context-based deep embeddings that effectively capture the semantic richness of textual attributes, making them particularly well-suited for addressing the challenges of node representations in TAGs. However, existing research often focuses on using LLMs solely as feature enhancement tools, overlooking the critical role of aggregators. These approaches often treat message passing as a fixed numerical process, without incorporating the semantic context of inter-node relationships. In comparison, we argue that LLMs are capable of much more than encoding text or classifying nodes—they can reason. Specifically, we leverage LLMs for **In-Context Relational Reasoning**, where the model interprets node relationships not merely as edges, but as semantic interactions. For example, in citation networks, a paper might cite another as a background source, a critical counterpoint, or an extension. Traditional GNNs cannot differentiate these subtle semantic differences. In contrast, our framework enables the LLM to read the textual attributes of the nodes involved, infer the nature of their relationship, and generate a tailored aggregation strategy (e.g., assigning lower or even negative weight to critical citations).

Traditional fixed aggregation operations, such as *sum*, *mean*, and *max*, struggle to capture the complex semantic relationships between neighboring nodes, particularly in scenarios where textual information is rich and diverse [14]. Notably, simple aggregation methods are prone to information loss, or *over-smoothing* [15,16], which ultimately reduces the accuracy of node representation. For example, Fig. 1 illustrates users' movie reviews within a social network, along with their spending on fan merchandise. In the movie recommendation scenario, users' reviews vary, often resulting in mixed opinions. Using a max or min aggregator may fail to accurately capture the overall preferences of the target users. In contrast, an average aggregator can better predict users' preferences for unrated movies, providing a more balanced measure of their interest. In a fan merchandise scenario, a user might make an unusually large purchase of the same item. If mean aggregation is applied, the impact of this anomalous transaction is diluted by the average of other normal transactions. In such cases, a max aggregator could help identify potential fraudulent activity by highlighting extreme behaviors. Therefore, when only a single GNN layer is used, different aggregators show clear deficiencies in distinguishing between different types of information, making it difficult to effectively handle complex node relationships and diverse information needs.

In this paper, we tackle a central challenge: “How can we design context-aware semantics-driven aggregation strategies that enable each node to selectively integrate information from its neighbors based on their relational meaning?” Rather than relying on static or shallow aggregation rules, we propose to leverage LLMs as in-context reasoners capable of interpreting the textual attributes of nodes and inferring the latent semantics behind their connections (e.g., supportive, critical, or neutral). These semantic reasoning are further used to guide the generation of tailored aggregation strategies for each node, transforming message-passing in GNNs into a truly semantically aware and context-sensitive process. To this end, we propose a novel method, LLM-based Tailored Node Aggregation (LTNA), which automatically generates semantic-aware aggregation strategies for each node, rather than relying on preset standard aggregators. Specifically, by using carefully designed task-specific instructions, we guide LLMs to generate natural language explanations and perform semantic reasoning without iterative training, thereby quickly building tailored aggregation strategies for each node. In LTNA, the LLM generates natural language interpretations based on the text information of the target node and its neighbors,

and infers specific aggregation behaviors accordingly. These aggregators can adaptively sample and weight aggregate neighbors with high semantic relevance to the target node, effectively suppressing the influence of irrelevant or noisy information. Ultimately, the embedding vectors of nodes are generated through these tailored aggregators, fully utilizing the capabilities of LLM in semantic modeling and context reasoning. This significantly improves the quality of representation learning and the performance of downstream classification tasks. Furthermore, the LLM generates customized aggregators for each node based on its textual information. These aggregators can adaptively sample and aggregate information from neighboring nodes that are closely related to the target node, ensuring the extraction of relevant information while minimizing interference from irrelevant data. The final node embeddings, produced through these tailored aggregators, leveraging the LLM's powerful semantic understanding and contextual analysis capabilities to capture complex inter-node relationships. This approach substantially enhances the quality of node representations and boosts performance in classification tasks. In summary, LTNA integrates the contextual reasoning ability of LLM with the graph structure information to enable a fully adaptive process-from semantic interpretation to aggregation strategy-bringing greater expressiveness and generalization ability to the representation learning in TAGs.

To comprehensively evaluate the effectiveness of our proposed LTNA approach, we conduct experiments on five widely used benchmark datasets and compare them against nine SOTA GNN models. The experimental results demonstrate that LLM-based tailored aggregator not only comprehends complex semantic information between nodes-going beyond individual word judgments-but also enhance robustness. In terms of specific metrics, LTNA achieves an average improvement of 7.95% in classification accuracy compared to the best baseline, while in the Micro-F1 metric, LTNA also achieves an average relative improvement of 9.90%. These results fully validate the robustness and broad applicability of LTNA across different datasets. The main contributions of this paper are summarized as follows:

- We propose a novel method, LTNA, to address the limitations of the fixed and uniform aggregation paradigm. By leveraging LLMs to encode textual attributes and aggregate features, richer semantic information can be extracted-going beyond simple numerical computations. This approach enhances robustness and reduces errors in attribute extraction.
- We guide the LLM to design tailored aggregation strategies for each node by optimizing the prompt. The aggregator adaptively samples and aggregates the information from neighboring nodes based on the target node's contextual information, effectively mitigating the issues such as over-smoothing or information loss commonly found in traditional aggregation methods.
- Experimental results on five widely used benchmark datasets demonstrate that LTNA consistently outperforms SOTA models across all evaluation metrics, underscoring the importance and necessity of tailored aggregation strategies for each node in complex network structures.

2. Related works

Representation learning on TAGs has garnered increasing attention in graph machine learning, with node classification being one of the most studied problems [17,18]. The problem can be formalized as a textual representation learning task, focusing on efficient learning and generation of embeddings through the textual features at each node. An early approach involves transforming textual attributes into shallow or hand-crafted features [7,8], the details of which are presented in Table 2. More recently, with the advent of Transformers [19] and pre-trained language models [20], these models have become the preferred choice for encoding contextual semantics in sentences for text representation learning. These encoded features can then be used as inputs to graph-based learning algorithms, such as GCN [21] or GAT [22]. These algorithms learn node embeddings and capture structural information about the graph while integrating extracted text features, achieving significant results. However, in scenarios involving complex semantic relationships and contextual information, relying solely on GNNs to make predictions using refined textual data is not sufficiently accurate, even though the node embeddings extracted through LLMs contain a large amount of information. This limitation arises because traditional GNNs adopt a fixed aggregation strategy, which struggles to fully capture the semantic complexity and richness of textual attributes, often focusing on only part of the information and leading to unsatisfactory overall performance.

Recently, many studies have focused on introducing LLMs into TAG tasks [23,24]. LLMs such as OpenAI's ChatGPT [25], Google's PaLM [11], and Meta's LLaMA [12] have demonstrated powerful semantic understanding and reasoning capabilities across a wide range of Natural Language Processing tasks. However, the potential benefits of LLMs in TAG tasks have not been systematically studied or deeply explored. LLMs not only can provide richer commonsense knowledge but also possess stronger semantic understanding and reasoning capabilities compared to shallow embedding, and this advantage is expected to enhance performance in TAG tasks. For example, GraphAlign [26] enhances the self-supervised pre-training effectiveness of text graphs by aligning cross-graph features, thereby obtaining stronger unified representations across multiple graphs. GRADMLP [27] embeds structural knowledge into text models through graph-aware distillation, enabling inference without explicit graph structures. GAUGLLM [18] leverages LLMs to generate high-quality features and structural augmentations, improving the training performance of graph contrastive learning and GNNs. Although some recent studies have attempted to utilize LLMs to improve node features or textual attributes [6,28], these works have rarely addressed the impact of different node aggregation strategies on the final results and thus have not achieved optimal performance. While these studies are undoubtedly valuable, they do not fully align with our specific research direction on TAGs. Our primary objective is to leverage the contextual understanding capabilities of LLMs to design tailored aggregation strategies for each node, enabling a more effective fusion of node textual information and graph structure to enhance task performance.

Different aggregators can significantly affect the final node's embedding representation during the message-passing process in GNNs. For instance, GraphSage [29] generates a node's embedding by learning a function that samples and aggregates the features of its local neighborhood, while GAT [22] aggregates neighboring information using learnable attention weights. Subsequent research

has further demonstrated that employing multiple aggregation strategies can effectively improve the performance of GNNs [30,31]. For example, Dehmamy et al. [32] have empirically found that using multiple aggregators (mean, sum, and normalized mean) to extract similar statistical information from input messages enhances the performance of GNNs in graph tasks. PNA [14] improves GNN performance by combining multiple aggregation functions, leveraging statistical features of neighborhood degree distributions, and adapting to different feature scales, thus increasing the expressiveness of the aggregator.

These studies highlight that aggregators have a crucial impact on GNN performance, and a single aggregator often fails to capture complex features effectively. However, most existing methods rely on fixed aggregator design and rarely design adaptive aggregators through the contextual information of each node. In complex graph networks, node features and neighbor information often exhibit significant differences, and a single aggregator is insufficient to distinguish between these features (as shown in Fig. 1), leading to suboptimal classification. Therefore, designing tailored aggregation strategies for nodes based on their contextual information is necessary to better capture the complex feature relationships in the graph and improve the overall GNN performance.

3. Preliminary

In this section, we introduce the notation and fundamental concepts used in our work, including the definition of TAGs, the large language model and prompt, and graph neural networks for node classification in TAGs.

Text-annotated graphs. Formally, a TAG can be represented as $\mathcal{G} = (\mathcal{V}, A, \{s_i\}_{i \in \mathcal{V}})$, where $\mathcal{V} = \{v_1, v_2, \dots, v_N\}$ is the set of nodes with $|\mathcal{V}| = N$, $A \in \mathbb{R}^{N \times N}$ denotes the corresponding adjacency matrix, and s_i is the sequential textual attribute associated with node i . We consider the task of classifying the nodes in the graph. In this paper, we focus on node classification for TAGs, one of the most commonly used graph-related tasks. Specifically, given some labeled nodes $B \subset \mathcal{V}$, the objective is to predict the labels of the remaining unlabeled nodes $U = \mathcal{V} \setminus B$.

Graph neural networks for node classification. Graph Neural Networks (GNNs) have recently been widely used to process graph-structured data [33]. The core idea of GNNs is to propagate node feature information through the graph's topology to perform node classification. The operation of GNNs can be briefly summarized in two steps: Neighbor Aggregation and Node Update. This process can be expressed as follows:

$$h_v^k = \text{UPD}(h_v^{k-1}, \text{AGG}(\{h_u^{k-1} : u \in \mathcal{N}(v)\})), \quad (1)$$

where h_v^k is the embedding of node v at the k th layer, and $\mathcal{N}(v)$ denotes the set of neighbors of node v . The functions **UPD** and **AGG** are typically differentiable, with **UPD** implemented as a neural network layer (e.g., a multilayer perceptron or an attention mechanism) and **AGG** responsible for aggregating the representations of the node's neighbors to produce an aggregated vector (e.g., mean, sum, and maximum). The final hidden embedding is fed into a fully connected layer for classification prediction.

Definition 1 (Aggregator). Let $\mathcal{G} = (\mathcal{V}, \mathcal{E})$ be an undirected graph with $\mathcal{V} = \{v_1, v_2, \dots, v_N\}$ as its vertex set and $\mathcal{E} \subseteq N \times N$ nodes in $\mathcal{G}$ as its edge set. For a node $v \in \mathcal{V}$ with neighbor set $\mathcal{N}(v)$, a function

$$\text{AGG} : \{h_u \mid u \in \mathcal{N}(v)\} \rightarrow \mathbb{R}^d$$

is called an **aggregator** if it merges the feature vectors h_u of the neighbors $u \in \mathcal{N}(v)$ into a vector in $\mathbb{R}^d$. Common examples include summation, averaging, and maximization.

Definition 2 (Distinguishable representation). We say that two node representations h_u and h_v are distinguishable if $h_u \neq h_v$. A set of node representations $\{h_1, h_2, \dots, h_N\}$ is fully distinguishable if, for any pair of distinct nodes $u, v \in \mathcal{V}$, their representations differ, i.e., $h_u \neq h_v$.

4. Methodology

In this section, we provide a detailed introduction of the proposed model LTNA. We first present an overview of tailored aggregators, followed by the two main modules—**structured prompting** and **adaptive aggregator generation**—which are central to the design of the tailored aggregator. Finally, we describe the training methodology of the model and the implementation of label prediction.

4.1. Overview of tailored aggregators

In this section, we introduce LTNA, an innovative LLM-based framework designed to address the inherent limitations of traditional GNNs through a highly tailored aggregation mechanism. By harnessing the superior contextual understanding and adaptability of LLMs, our approach extracts richer semantic information, rather than relying solely on simple numerical computations. This significantly enhances the expressive power of GNNs while improving model robustness, reducing errors during attribute extraction, and overcoming the rigidity of conventional aggregation methods that rely on fixed strategies such as sum, mean, or max. These static approaches lack the flexibility to adapt to the dynamic and diverse needs of individual nodes, leading to information loss, oversimplification, and reduced accuracy—ultimately limiting their ability to handle complex real-world graph.

To address these challenges, we first introduce a **structured prompting technique** that allows the LLM to adaptively identify and sample contextually relevant neighboring nodes. This context-aware neighbor sampling is crucial, as nodes assign varying importance to their neighbors' information based on specific contexts. Subsequently, we present an **adaptive aggregator generation** mechanism within LTNA, which dynamically adjusts aggregation strategies to capture the complex relationships between nodes and

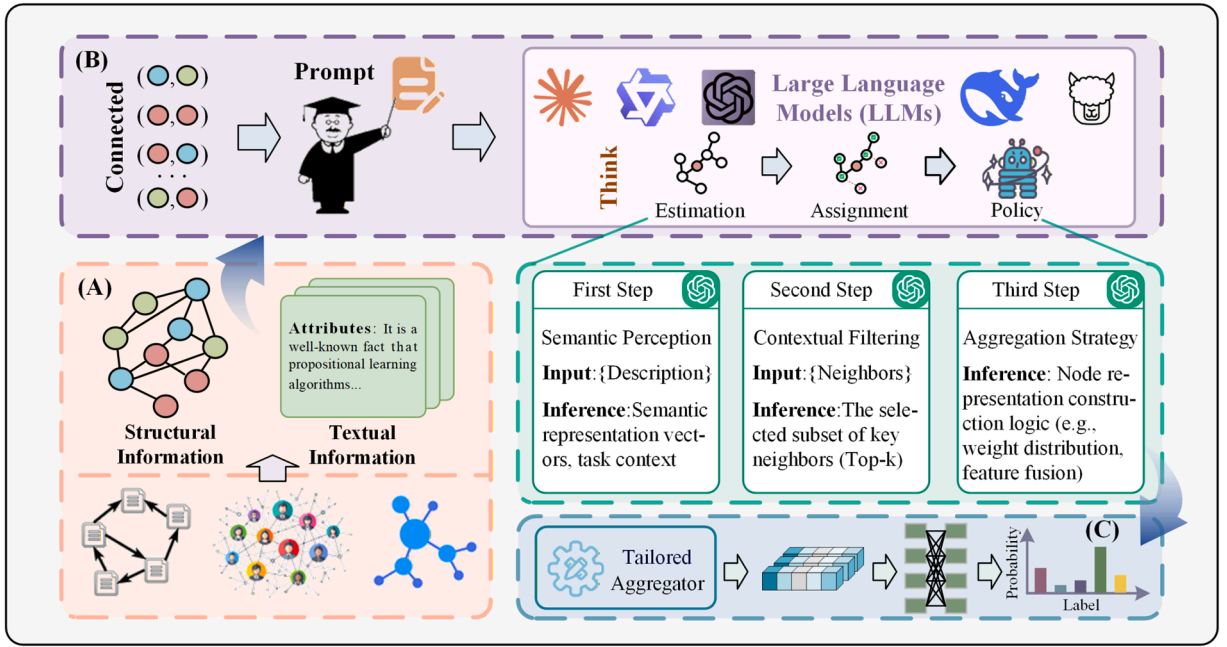


Fig. 2. Our framework utilizes LLMs to generate tailored context-aware aggregators for each node to enhance the representation learning of TAGs. The process begins with the acquisition of TAGs, followed by integrating the topological and textual information into a customized prompt. This prompt is then used to guide the LLM, in this case, LLaMa3-8B, which generates a novel aggregation mechanism. Finally, the input data is messaged through the resulting aggregator to obtain rich node features, which are subsequently mapped to obtain the labeled probability distribution of the nodes.

their neighbors. As a result, LTNA significantly enhances both the expressiveness and the accuracy of node representations. Finally, we introduce a **unified model training** methodology that incorporates the label prediction for both model training and downstream task inference. This seamless adaptation to the semantic and structural intricacies of nodes leads to smoother and more precise representations, particularly in complex graph networks.

Theorem 1 (Minimum number of aggregators required). *Let $\mathcal{G} = (\mathcal{V}, \mathcal{E})$ be a graph with $|\mathcal{V}| = N$. In order to produce fully distinguishable representations for all N nodes in $\mathcal{G}$, one needs at least k distinct aggregators in the chosen class of aggregators.*

Proof. Assume that only $N - 1$ aggregators, i.e., $g_1, g_2, \dots, g_{N-1}$ are used. These aggregators must be capable of distinguishing all multisets of size N . This implies that there exist multisets that share the same aggregator g_k .

Define a continuous mapping $f: S \rightarrow \mathbb{R}^{N-1}$, where S represents the collection of multisets. The mapping f assigns each multiset X to a vector in $\mathbb{R}^{N-1}$, where each coordinate corresponds to the output of one of the $N - 1$ continuous aggregators:

$$f(X) = (g_1(X), g_2(X), \dots, g_{N-1}(X)),$$

since we assumed these aggregators are able to discriminate between all the multisets, f is injective.

As S is a n -dimensional Euclidean subspace, it is possible to define an $(N - 1)$ -sphere C^{N-1} entirely contained within it, i.e., $C^{N-1} \subseteq S$. According to the Borsuk-Ulam theorem [34,35], for continuous function $f: C^{N-1} \rightarrow \mathbb{R}^{N-1}$, there must exist two antipodal points that $\vec{x}_1$ and $\vec{x}_2$ satisfy:

$$f(\vec{x}_1) = f(\vec{x}_2).$$

This implies that at least two distinct multisets become indistinguishable under the mapping f , meaning that f cannot be injective, thereby leading to the desired contradiction.

Unlike previous approaches [14] that filter or combine from a set of existing aggregators, our method leverages LLMs to analyze node context and generate a tailored aggregator aligned with both the node's information and that of its neighbors. $\square$

4.2. Design of tailored aggregators

To implement the above concept of tailored aggregators, we propose the LTNA method. This method leverages the powerful context-awareness and superior semantic understanding of LLMs to generate tailored aggregators for each node in the graph, thereby enhancing the node representation learning capability of the model. The overall process is illustrated in Fig. 2.

Structured prompt. We first design structured prompt $p(v)$ based on the textual information of the nodes. For example, in a citation network, $p(v)$ can be represented as $t_v, a_v, \mathcal{N}(v), \dots$, where t_v and a_v refer to the paper title and abstract of the text information for

node v , respectively, and $\mathcal{N}(v)$ represents the neighbor information of node v . $p(v)$ guides the LLM to perform an in-depth analysis of the semantic content of neighboring nodes, ensuring that the generated aggregator can flexibly adapt to the semantic context of the nodes rather than relying solely on a fixed graph structure.

$$p(v) = \text{Prompt}(\text{prefix}, s_v, s_u \mid u \in \mathcal{N}(v)). \quad (2)$$

Moreover, $p(v)$ denotes the prompt of node v used to guide the LLM. Its instructions aim to generate personalized semantic aggregators for each node while minimizing interference from irrelevant information during aggregation. This design helps ensure that the generated aggregators accurately capture the key associative relationships between nodes and their neighbors. Let the textual feature of node v be s_v , the set of neighboring nodes be $\mathcal{N}(v)$, and the set of textual features of the neighbors be $S_{\mathcal{N}(v)} = \{s_u \mid u \in \mathcal{N}(v)\}$. We define the neighbor selector generated by the LLM as:

$$\mathcal{N}'(v) = \text{Select}_{\text{LLM}}(s_v, S_{\mathcal{N}(v)}) \subseteq \mathcal{N}(v), \quad (3)$$

where $\text{Select}_{\text{LLM}}$ is a subset selection operation by the LLM based on semantic similarity, task context, textual coherence, and other reasoning strategies. To control complexity, the number of selected neighbors is limited to a maximum of k , which is typically set according to the degree of the average node. Since some neighboring nodes may convey irrelevant or even misleading information, this selection process is essential for enhancing representation quality and preventing semantic noise.

Adaptive aggregator generation. LLMs demonstrate strong context awareness and superior semantic understanding through pre-training on large-scale text corpora. We design a specific prompt $p(v)$ to feed into LLMs, enabling them to generate tailored aggregators for each node in the graph based on the node's semantic context. Most importantly, LLMs are robust to textual variations and can effectively summarize overall semantics while remaining less susceptible to interference from individual anomalous or misleading words or numbers. This ensures the stability of the aggregation results.

In our implementation, the tailored aggregators $\text{TAGG} = \{g_v \mid v \in \mathcal{V}\}$ are not conventional analytical aggregation functions (e.g., mean, sum, or max), but rather a semantic policy dynamically produced by the LLM based on the contextual relationships, semantic relevance, and task objectives of graph text tokens. First, the LLM analyzes the textual context of the target node and its neighbors to interpret their semantic relationships. Then, based on this explanation, the LLM generates an actionable aggregation strategy, such as assigning weights to neighbors or selecting a specific aggregation rule. Formally, this process is as follow:

$$\mathcal{E}_v = \text{Explain}_{\text{LLM}}(p(v), S_{\mathcal{N}'(v)}), \quad (4)$$

$$g_v = \text{Act}_{\text{LLM}}(\mathcal{E}_v), \quad (5)$$

where $\mathcal{E}_v$ is represented as an explanatory operation by the LLM. This explanation determines how to integrate information from selected neighbors based on their semantic relationships with the target node. For example, the LLM may output: "Focus on neighbors that discuss GNNs and ignore those about NLP, then average their representations." g_v is a semantics-aware aggregator that adaptively selects and integrates neighbor information based on contextual relevance. This allows the aggregator to flexibly select the most appropriate aggregation method according to the individual needs of each node, rather than relying on fixed rules. Unlike traditional deep learning methods that require iterative training of aggregator parameters, LTNA utilizes a pre-trained LLM to generate a tailored aggregation policy for each node through a single forward inference, thereby eliminating the need for explicit aggregator training. Although LTNA does not require additional optimization for the aggregator, the computational cost remains nontrivial due to the inference overhead introduced by the LLM. Thereby, the efficiency gain of LTNA lies primarily in avoiding the training process rather than in substantially reducing overall runtime.

In our framework, the LLM is responsible for two core tasks: determining which neighbors to aggregate ("Who to aggregate") and designing how to aggregate them ("How to aggregate"). Then the aggregation strategy and the neighbor selection are jointly expressed as follows:

$$(\mathcal{N}'(v), g_v) = \Pi_{\text{LLM}}(p(v)), \quad (6)$$

$$\Pi_{\text{LLM}} \triangleq \left\{ \pi_v = \text{Act}_{\text{LLM}}\left(\text{Explain}_{\text{LLM}}(p(v), \text{Select}_{\text{LLM}}(p(v)))\right) \right\}, \quad (7)$$

$$\text{Reach}(\Pi_{\text{LLM}}) \subseteq \text{TAGG}, \quad (8)$$

where $\text{Reach}(\Pi_{\text{LLM}})$ represents the aggregate reachable set induced by the policy Π_{LLM} . Because it covers a richer class of aggregation functions, it is expected to provide stronger node discriminability and greater robustness to noise than fixed aggregators, thereby inducing an executable aggregation behavior $g_v \in \text{TAGG}$. This process is a forward inference and does not involve fine-tuning.

Finally, by using the generated tailored aggregator to message the sampled most valuable neighbor nodes. The generated embeddings h_v are able to accurately reflect the semantic relationships between nodes and their neighbors, which greatly improves the quality of node embeddings:

$$h_v = g_v(\{h_u, h_u \mid \forall u \in \mathcal{N}'(v)\}). \quad (9)$$

With the semantics-aware aggregator generated by the LLM and adaptive selection of neighbor information based on context, we not only improve the flexibility and accuracy of the aggregation process but also reduce the need for iterative model training. The generated node embeddings are able to better capture complex relationships between nodes, thus improving the overall accuracy of the task.

In summary, this paper proposes a novel aggregation method that inputs a carefully designed prompt into the LLM, which the generates an adaptive aggregator based on the semantic context of each node. This method aggregates the most relevant neighbor

information through sampling, overcoming the limitations of traditional fixed aggregation approaches and significantly improving the quality of node embeddings with a semantically-aware sampling strategy. Ultimately, this approach leads to better performance in node classification tasks.

4.3. Training & prediction

Once the LLM-generated tailored aggregator is applied to the token representations, the aggregated result is used as the node embedding $h_v \in \mathbb{R}^d \mid v \in \mathcal{V}$. We pass it through a task-specific classifier $f_\theta : \mathbb{R}^d \rightarrow \mathbb{R}^{|\mathcal{Y}|}$ to obtain the probabilistic label formulated as:

$$\mathcal{Y}_v = f_\theta(h_v) = \sigma(\mathbf{W}h_v + b), \quad \text{for all } v \in \mathcal{B}. \quad (10)$$

where h_v is the context-aware embedding generated by the LLM-guided aggregator, $\mathbf{W}$ and b are trainable parameters, and $\sigma(\cdot)$ is a non-linear activation function (e.g., ReLU). The model is trained using a cross-entropy objective defined on the labeled training nodes $\mathcal{B} \subset \mathcal{V}$:

$$\mathcal{L}_{\text{task}}(\theta) = \sum_{i=1}^{|\mathcal{B}|} \mathbb{E}_{\pi_i \sim \Pi_{\text{LLM}}} \mathbb{E}_{y_i \sim \mathbf{Y}} [\mathcal{L}(f_\theta(\pi_i(\mathcal{N}'(v_i))), y_i)], \quad (11)$$

where y_v is the true label of node v , $\mathbf{Y}$ denotes the ground-truth labels, $\mathcal{L}(\cdot, \cdot)$ represents the cross-entropy loss. To prevent overfitting and control the complexity of model, we add an L_2 regularization term to the loss function, penalizing large weights in the classifier:

$$\mathcal{L} = \mathcal{L}_{\text{task}}(\theta) + \lambda \|w\|_2^2, \quad (12)$$

where $\|w\|_2^2$ denotes the sum of squared Frobenius norms of the classifier's weights, and λ is a regularization coefficient.

In our framework, only the classifier parameters w are updated during training. The LLM used to generate the neighbor selection and aggregation functions remains frozen, and its outputs are computed once per node, enabling a plug-and-play integration without backpropagation through the LLM.

4.4. Discussion

To the best of our knowledge, LTNA is the first research effort to incorporate LLM into GNN architectures for reconfiguring aggregation logic based on semantic and contextual reasoning. Unlike previous approaches that primarily focus on improving node encoders or task-specific prediction heads, LTNA embeds the in-context reasoning capability of LLMs directly into the message-passing stage of GNNs. Specifically, LTNA prompts the LLM to generate a tailored aggregation strategy for each node, based on its own attributes and the textual descriptions of its neighbors. This unifies neighbor selection and aggregation function design into a single, LLM-controlled process-yielding task-aware and semantics-driven information integration.

Importantly, LTNA does not require fine-tuning the LLM on graph data. Instead, it operates in a zero-shot manner, leveraging the LLM's pretrained world knowledge and reasoning ability to synthesize a unique aggregation policy for every node. From a meta-learning perspective, the LLM acts as a meta-learner that, given a task-specific prompt (i.e., a node and its neighborhood context), generates a one-time learner (i.e., the aggregator). This represents a form of in-context meta-learning, eliminating the need for complex meta-training pipelines typically used in neural architecture search or few-shot graph learning.

Compared to classical attention-based models like GAT, which perform adaptive weighted averaging, LTNA offers significantly greater expressive power. GAT can adjust “who to listen to” and “how much,” but its aggregation operation remains a fixed weighted sum, limiting its ability to capture relational semantics or adapt the functional form of aggregation. In contrast, LTNA enables not only neighbor weighting but also aggregation mode selection and even the generation of new, nonlinear rules. For instance, it can learn to ignore irrelevant neighbors, apply max pooling for anomaly detection, or construct symbolic expressions such as $(\text{node}_A + \text{node}_B) \cdot 0.8 - \text{node}_C \cdot 0.5$, depending on the inferred context-capabilities that are fundamentally out of reach for GAT architectures.

This transition, from parameter-weighted aggregation to semantics-driven reasoning, marks a paradigm shift in graph representation learning. By harnessing the LLM's reasoning abilities, LTNA redefines the role of aggregation: from a fixed computational primitive to a dynamic, interpretable, and context-aware reasoning process.

5. Experiments

In this section, we first introduce the dataset and experimental setup. Next, we evaluate the effectiveness of the LLM-based aggregator in improving the performance of the GNN to address the following research questions:

- **Q1:** What is the performance of LTNA compared to SOTA graph algorithms? (Section 5.2)
- **Q2:** How does the each component of LTNA contribute to classification results? (Section 5.4)
- **Q3:** Do the aggregators generated by the LLM enhance the effectiveness of the embeddings? (Sections 5.2 & 5.5)
- **Q4:** How does the number of sampled neighbors influence the accuracy of the model? (Section 5.6)

Table 1
Detailed statistics of the five evaluation datasets.

Dataset	#Nodes	#Edges	#Label	#Metric	#Avg. Degree
CORA	2708	5429	7-classes	Accuracy, Macro-F1	4.01
CITeseer	3186	4277	6-classes	Accuracy, Macro-F1	2.68
PUBMED	19,717	44,338	3-classes	Accuracy, Macro-F1	4.50
OGBN-ARXIV	169,343	1,166,243	40-classes	Accuracy, Macro-F1	13.67
OGBN-PRODUCTS (SUBSET)	54,025	74,420	47-classes	Accuracy, Macro-F1	5.58

Table 2
Details of text preprocessing and feature extraction methods used for TAGs.

Dataset	Methods	Features	Description
CORA	BoW	1433	After stemming and removing stopwords, there are 1433 unique words that appear at least ten times in these papers.
CITeseer	BoW	3703	After stemming and removing stopwords, the resulting vocabulary consists of 3703 unique words. All words with a document frequency of less than ten were removed.
PUBMED	TF-IDF	500	Each publication in the dataset is represented by a TF/IDF weighted word vector derived from a dictionary containing 500 unique words.
OGBN-ARXIV	skip-gram	128	The embeddings of individual words are computed using the skip-gram model on the Microsoft Academic Graph corpus.
OGBN-PRODUCTS	BoW	100	Node features are generated by extracting BoW features from product descriptions and then performing Principal Component Analysis to reduce the dimensionality to 100.

5.1. Experimental setup

Datasets. In the experiments, we utilize five benchmark datasets for node classification: CORA [36] is a citation dataset with 2708 scientific publications (nodes) categorized into seven categories, connected by 5429 citation edges. Each paper cites or is cited at least once by another paper. CITeseer [37] is another citation dataset in which TAG contains 3186 nodes, categorized into six fields: AI, ML, IR, HCI, DB, and agents. The graph consists of 4277 edges representing referential relationships between nodes. PUBMED [38], similar to the previous two datasets, is a citation dataset with 19,717 diabetes-related papers categorized into three types and connected by 44,338 citation edges. OGBN-ARXIV [39] is a directed citation graph of 169,343 arXiv CS papers with 1,166,243 edges. Each paper cites others, and the task is to predict the 40 subject areas to which these arXiv papers belong (e.g., CS.AI, CS.LG). A detailed description of this dataset can be found in¹. OGBN-PRODUCTS [39] is an undirected co-purchase graph of 2,449,029 Amazon items and 61,859,140 edges. Given our computational and academic resource constraints, we conduct our evaluation on sampled subgraphs. Specifically, we construct a subgraph with approximately 54K nodes and 74K edges for training and testing using a node sampling strategy. The task is to predict product categories through multi-class categorization, using 47 top-level categories as target labels. Detailed statistics and descriptions are provided in Table 1.

Earlier approaches typically extracted node features from the TAGs by transform text attributes into shallow or hand-designed features for use in GNN models for node classification tasks. Table 2 summarizes the text preprocessing and feature extraction methods widely used in current popular GNN research in graph libraries such as PyG and DGL. While these methods are simple to implement and computationally efficient, they rely on traditional language modeling techniques, making it difficult to effectively capture the contextual information of polysemous words and resulting in a shallow understanding of text semantics. This limitation can reduce the expressive power of extracted node features, which in turn negatively impacts the performance of downstream tasks.

Following previous works [17,40], we adopt a data split where 60% of the nodes are used for training, 20% for validation, and 20% for testing in the CORA, CITeseer, and PUBMED. For OGBN-ARXIV and OGBN-PRODUCTS, we use their official splits. Due to resource constraints, we conducted the OGBN-PRODUCTS experiments using subgraph samples.

Implementation. For all experiments, we run the models for 100 epochs in each evaluation process, using ReLU as the activation function and Adam as the optimizer, where the learning rate is 0.01. We employ a 3-layer MLP with hidden sizes of 128 and 64 in our module and set the dropout rate to 0.5. All GNNs are carefully tuned according to our dataset splits, ensuring relative fairness in the experiments. Additionally, we use classification Accuracy and Macro-F1 as evaluation metrics. The model's performance is optimized using the Adam optimization algorithm. In addition, in order to prevent over-fitting, the Dropout technique is introduced into the model to improve the model's generalization ability by randomly dropping some neurons.

Baselines. We compare LTNA with thirteen classical and advanced baseline methods to validate the effectiveness of the proposed approach, including (1) MLP; (2) general GNN models: GCN [21], GAT [22], GraphSAGE [29], and MixHop [41]; (3) augmentation models: APPNP [42] combines personalized PageRank with neural networks to develop a novel, improved propagation scheme. GPR-GNN [43] can adaptively learn Generalized PageRank (GPR) weights to jointly optimize node feature extraction and topological information. RNCGLN [44] improves the expressiveness of node representations by designing a graph contrastive loss for local graph learning and employing self-attention for global graph learning. GCN+ReP [45] is the GCN-based model with the reverse process,

¹ <https://ogb.stanford.edu/docs/nodeprop/>.

Table 3

The classification *Accuracy* ($\uparrow$) and *Macro-F1* (%) ($\uparrow$) across all methods for the CORA, CITESEER, PUBMED, OGBN-ARXIV, and OGBN-PRODUCTS datasets. We use BoldUnderline to denote the best performance, *Italic* the second best one, **Bold** the third best one, and “–” refers to out-of-memory. The last row in the table indicates the improvement of our method relative to the best baseline performance in the same dataset.

Model	CORA		CITESEER		PUBMED		ARXIV		PRODUCTS	
	Acc.	F1	Acc.	F1	Acc.	F1	Acc.	F1	Acc.	F1
MLP	74.22	72.12	71.17	68.59	87.32	87.25	52.25	27.24	53.38	21.69
GCN	84.90	82.73	72.52	69.62	86.69	86.14	64.28	37.02	66.35	28.85
GAT	83.43	81.87	73.42	70.28	85.47	84.68	62.89	28.91	66.46	28.22
GraphSAGE	86.37	84.55	74.92	71.85	88.13	87.68	63.91	34.39	65.75	28.15
Mixhop	84.71	82.83	72.67	69.27	88.27	87.89	68.32	43.41	69.34	30.44
IDGL	86.92	81.66	74.46	68.92	88.32	87.18	71.57	48.92	68.02	29.51
GraphNAS	87.08	83.85	75.67	69.02	88.58	87.41	64.23	36.85	67.18	28.42
APPNP	86.19	84.22	74.02	71.62	87.42	86.79	62.94	30.92	69.33	29.56
GPR-GNN	86.04	84.02	73.72	70.70	88.34	87.87	66.04	41.44	68.18	30.05
RNCGLN	87.02	85.51	77.18	72.84	86.99	86.80	–	–	–	–
GCN + ReP	86.88	85.97	74.23	68.67	89.91	89.74	60.26	38.36	71.24	32.62
LTNA (Ours)	<u>91.04</u>	<u>89.68</u>	<u>85.24</u>	<u>80.56</u>	<u>92.95</u>	<u>91.31</u>	<u>76.36</u>	<u>50.42</u>	<u>81.71</u>	<u>42.33</u>
	+3.96	+3.71	+8.06	+7.72	+3.04	+1.57	+4.79	+1.50	+10.47	+9.71

generating distinguishable node representations by inverting forward message propagation; IDGL [46] achieves strong performance by alternately optimizing the graph structure and node representations under similarity-based objectives and adaptive regularization. GraphNAS [47] is a reinforcement learning-based GNN architecture search method that can automatically generate and optimize GNN structures. (4) LLM-based enhancement model: TAPE [40] proposes a novel method that leverages an LLM to generate explanation text and transform it into GNN features to improve node classification performance on TAGs. GLEM [48] proposes an efficient method based on the variational Expectation-Maximization framework to jointly model and infer TAGs by alternately training LLMs and GNNs. GraphEdit [49] fine-tunes LLMs with task instructions to learn complex node relationships in graphs, enabling robust structure learning and denoising without relying on explicit structural supervision. OFA [50] transforms graph structures and attributes into natural language and introduces a node-level attention mechanism with a graph cueing strategy. GraphAlign [26] achieves cross-graph feature alignment through a hybrid module combining feature encoding, normalization, and feature experts. GRADMLP [27] embeds structural knowledge into text models through graph-aware distillation, enabling inference without explicit graph structures.

5.2. Results and cost analysis

In this subsection, we validate the effectiveness of the proposed model by comparing it with current popular GNN algorithms. In general, these GNN algorithms use input data where the textual attributes of the dataset are transformed into shallow or hand-crafted features, such as skip-gram [7] or BoW [8] features, followed by message passing through a fixed aggregation strategy. In contrast, our model leverages the semantic understanding capability of the LLM to design adaptive aggregators for the textual attributes of each node in the dataset, thus achieving more refined node aggregation. The experimental results, shown in Table 3, demonstrate that the aggregator plays a crucial role in the node information aggregation process.

Our model uses the LLM’s semantic analysis of node text attributes to generate specific aggregators for each node, allowing it to capture the complex relationships among nodes. Additionally, to achieve optimal performance within limited experimental resources, we leverage the LLM’s contextual understanding to select the most influential neighboring nodes for each target node and aggregate information based on the generated aggregators. This results in smoother and more expressive node representations.

The experimental results show that our method consistently outperforms other comparative methods across all datasets, fully validating the feasibility and effectiveness of generating node aggregators through LLM. In the five datasets, our model achieves relative improvements over the best baseline method in terms of accuracy by 4.55%, 10.44%, 3.38%, 6.69%, and 14.70%, and in terms of Macro-F1 scores by 4.32%, 10.60%, 1.75%, 3.07%, and 29.77%, respectively. These results indicate that the LTNA method is able to effectively enhance the performance of the node classification task by employing a differentiated aggregation strategy tailored to each node based on complex graph network data.

To quantitatively assess the resource consumption and runtime efficiency of LTNA, we provide detailed profiling results encompassing LLM inference latency, average token length, GPU and CPU memory utilization, and average node throughput. All measurements were obtained under the identical hardware configuration, and the results are summarized in Table 4.

5.3. Comparison with concurrent work

In this section, we explore in detail the impact of different text embedding methods on node feature enhancement and compare them with our proposed model, which uses prompting to verify the effectiveness of the proposed method. For this purpose, we select three widely used text embedding methods: the lightweight deep text embedding model Sentence-BERT [51], the Bert-based pre-trained model SCIBERT [52], specifically designed for scientific texts, the online sentence embedding model Palm-Cortex-001

Table 4

Profiling results of LTNA across benchmark datasets. All profiling results are obtained using the default maximum context length of LLaMa3-8B, with batched inference under limited concurrency on a server equipped with $4 \times$ Intel(R) Xeon(R) Gold 5220 CPU @ 2.20GHz, $4 \times$ NVIDIA L40 GPUs, and 512 GB RAM.

Datasets	Latency	Average Token	GPU Memory	CPU Memory	Throughput
CORA	0.14	698.8	14.6	27.7	6.67
CITSEER	0.13	653.7	14.5	23.5	7.16
PUBMED	0.26	1420.0	15.4	24.1	3.70
ARXIV	0.31	1645.5	15.59	134.98	3.16
PRODUCTS	0.10	437.0	22.3	34.4	10.17

Table 5

The classification *Accuracy* and *Macro-F1* (%) across all methods for the CORA, PUBMED, and OGBN-ARXIV datasets.

Model	CORA		PUBMED		OGBN-ARXIV	
	Acc.	F1	Acc.	F1	Acc.	F1
Sentence-BERT + MLP	81.95	78.96	89.73	89.59	70.34	40.68
Sentence-BERT + GCN	88.79	86.92	88.08	87.73	72.78	42.74
SCIBERT + GCN	89.16	87.97	87.80	87.55	71.21	48.83
Palm + MLP	80.66	76.92	90.05	89.81	70.39	41.74
Palm + GCN	88.49	86.75	88.36	87.71	72.39	43.53
LLaMa 8B + MLP	83.61	82.36	85.64	84.48	71.78	43.82
LLaMa 8B + GCN	85.82	84.02	86.74	86.24	73.61	44.81
TAPE	89.53	87.38	93.02	91.29	75.35	48.36
GLEM	88.49	87.16	92.56	90.68	73.57	46.56
GraphEdit	90.03	88.79	92.87	90.89	74.89	47.10
OFA	77.69	75.52	80.43	79.79	74.81	46.89
GraphAlign	85.90	83.75	89.72	85.74	75.30	47.62
GRADMLP	82.02	79.62	87.73	87.16	71.89	44.94
LTNA (Qwen3 4B)	89.32 $\pm$ 0.94	88.04 $\pm$ 0.86	89.70 $\pm$ 1.23	89.12 $\pm$ 1.33	73.92 $\pm$ 0.57	45.46 $\pm$ 1.14
LTNA (LLaMa3 8B)	91.04 $\pm$ 1.24	89.68 $\pm$ 1.15	92.95 $\pm$ 1.14	91.31 $\pm$ 1.13	76.36 $\pm$ 0.39	50.42 $\pm$ 0.33
LTNA (Qwen3 14B)	92.13 $\pm$ 0.52	91.37 $\pm$ 0.60	92.59 $\pm$ 0.70	91.05 $\pm$ 0.68	77.85 $\pm$ 0.38	49.61 $\pm$ 0.73
LTNA (Qwen3 32B)	92.70 $\pm$ 0.91	91.75 $\pm$ 0.99	92.86 $\pm$ 0.81	91.27 $\pm$ 0.79	78.56 $\pm$ 0.59	49.23 $\pm$ 1.16

introduced by Google [53], and the open-source large language models LLAMA-ccp² and Qwen [54], which have gained widespread adoption. By designing effective prompts, we guide the LLM to generate higher-quality node embeddings and create tailored aggregators for each node to improve the overall performance. The experimental results are presented in Table 5.

In our experiments, we use MLP and GCN as predictors, respectively. First, MLP is used to evaluate the quality of text embeddings without aggregation. The results show that although LLAMA demonstrates strong language comprehension, its embeddings generated without prompts fail to achieve the expected results in the classification task, with a significant performance gap compared to deep text embedding models like Sentence-BERT and Palm-Cortex-001. This suggests that the base embeddings from language models alone may not be sufficient to capture the complex semantic features of nodes. Next, the performance of all models improved when using GCN as a predictor, further validating the critical role of aggregated neighbor information for the node classification task. However, the performance of some models showed degradation, possibly due to the introduction of irrelevant or noisy information during the aggregation process. This phenomenon highlights the importance of aggregator design in effective information fusion.

Recognizing the importance of aggregators in GNNs, we design a tailored aggregator for each node using prompt, allowing the model to dynamically adjust the aggregation strategy based on the node’s contextual information. Experimental results show that the model with prompting achieves the best performance among all tested approaches. This not only demonstrates the effectiveness of the prompt design but also shows that the tailored aggregator helps the model better capture the complex relationships between nodes and their neighbors, thus improving overall performance. We conducted additional experiments comparing three backbone models of varying sizes: Qwen3-4B, LLaMA3-8B, Qwen3-14B, Qwen3-32B reporting the average classification accuracy and standard deviation across 10 different random seeds. Although Qwen3-32B achieved slightly better results than LLaMA3-8B on some datasets, its computational cost was substantially higher, leading to a low cost-effectiveness ratio (performance gain vs. resource consumption). Therefore, we retained LLaMA3-8B as the main backbone model, as it demonstrates the effectiveness of the strategy mechanism while maintaining an optimal balance between performance and computational efficiency.

² <https://github.com/ggerganov/llama.cpp>.

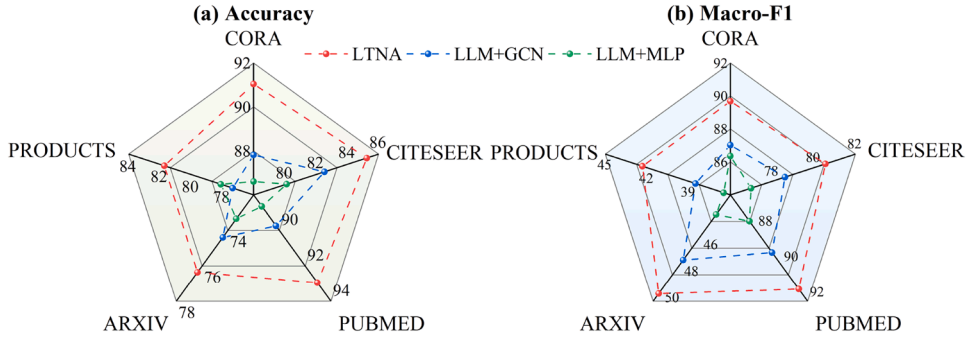


Fig. 3. Ablation study on all datasets illustrates the impact of the aggregator design on the classification performance, where LLaMa3-8B is used as the LLM with prompt input.

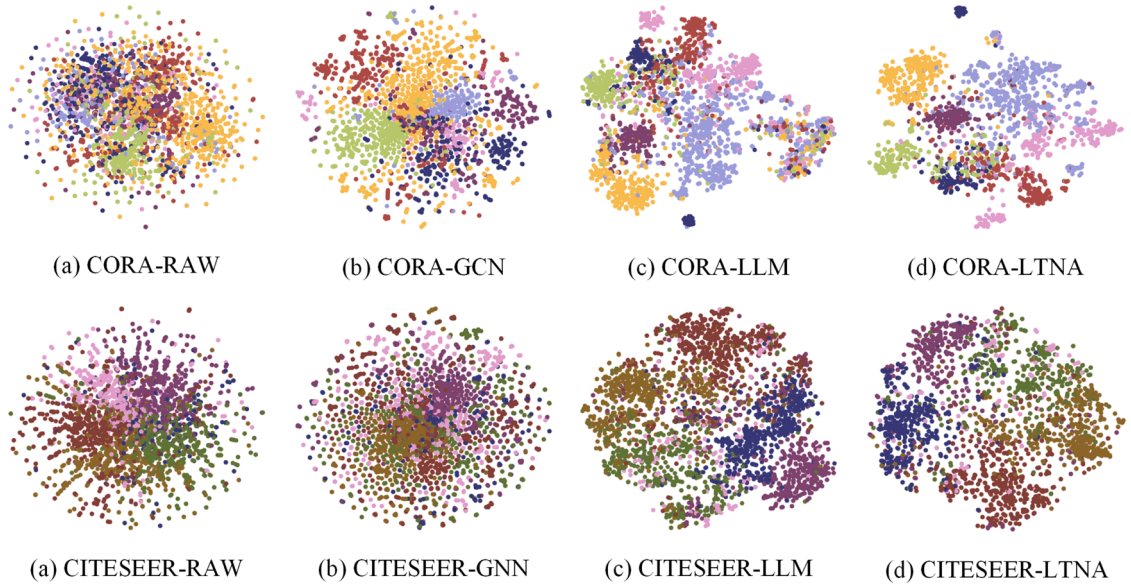


Fig. 4. The visualization of node embedding for CORA and CITESEER. Each column sequentially presents: (a) the original node feature embeddings; (b) the embeddings after aggregating the original features using GCN; (c) the embeddings obtained by encoding the textual attributes of the nodes using LLM, (d) the embeddings generated after designing adaptive aggregators for each node via LLM.

5.4. Ablation study

We conduct a comprehensive ablation study to assess the importance of each design component in the proposed framework across five datasets. Specifically, the following ablative baselines are considered: LLM + GCN which uses LLM as a textual encoder to capture the semantic representation of the text only, subsequently applies GNNs to the refined textual data to generate predictions. However, it employs a fixed aggregation strategy; LLM + MLP, which replaces the GCN with an MLP that operates on the textual representation processed by the LLM for label prediction. The key difference from our approach is that it does not incorporate any aggregation strategies. The results are summarized in Fig. 3.

From the results, we first observe that GNNs consistently exhibit ubiquitous advantages in handling the node classification tasks on TAGs. Even when leveraging the high-quality textual representation provided by LLMs, the LLM + MLP baseline demonstrates extremely inferior performance in terms of both accuracy and Macro-F1 scores. This reveals that shallow embedding methods struggle to capture complex semantic relationships and fail to fully exploit the richness of textual attributes. Moreover, the comparison with the LLM + GNN baseline underscores the critical role of our proposed tailored aggregator generation in LTNA. When using a fixed aggregator, the LLM + GNN baseline performs noticeably worse than our method with adaptive aggregators across all five datasets. This validates the need for distinct aggregators tailored to each node to effectively capture complex relationships between neighboring nodes in complex graph networks. In conclusion, the ablation study provides valuable insight that the ability to generate context-aware, adaptive aggregators plays a pivotal role in enhancing node classification performance, elevating it to state-of-the-art levels.

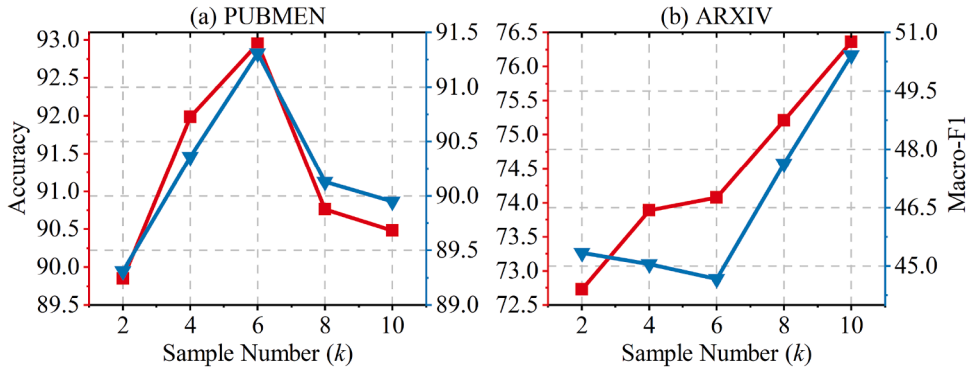


Fig. 5. Sensitivity analysis results for LTNA with varying numbers of sampled neighbors. We present the *accuracy* and *macro-F1* scores evaluated on PUBMEN and ARXIV.

5.5. Visualization

To intuitively access whether the aggregator generated by the LLM in the proposed method effectively improves the quality of node embeddings, we employ the t-SNE method to visualize node embeddings from the following four models in a 2D space: (a) raw features obtained after the shallow embedding mapping; (b) node representation learned through a GNN with a convolutional aggregation strategy; (c) embeddings obtained after encoding the nodes' textual attributes using the LLM; and (d) embeddings generated via adaptive aggregators for each node using the LLM. In the visualizations, nodes with the same color represent the same category, and if nodes of the same color are clustered while nodes of different colors remain separated, it suggests the embedding quality is effective.

The results presented in Fig. 4 reveal several key observations. First, we observe that in the first two columns, the class embeddings appear relatively mixed, with nodes scattered and categories blending together without forming clear clusters. Although the GNN embedding shows slight improvement over the raw features, distinct category clusters are still not evident. This highlights the limitation of using a vanilla GNN for the node classification tasks, demonstrating its relatively weak representation learning capability. It further emphasizes the need for more advanced representation learning techniques and customized aggregation functions.

The third and fourth columns illustrate the impact of using the LLM to enhance node representations. Specifically, the third column shows that node embedding quality is significantly improved compared to both raw and GNN embeddings, indicating that hand-designed features alone fail to capture the full semantic richness of the nodes adequately. Finally, the fourth column underscores the superiority of adaptive aggregators generated by the LLM. Notably, in the 2D space, nodes of different categories are well-separated, and nodes within the same category cluster tightly, forming clear groupings. This demonstrates the effectiveness of LLM-generated aggregators in improving node representation learning and highlights the necessity of designing tailored aggregation strategies to account for complex relationships between nodes.

5.6. Sensitivity analysis

In this subsection, we explore the impact of a key hyperparameter of LTNA, i.e., the number of neighboring nodes sampled. As discussed in Section 5.5, the node embeddings generated by the proposed method are significantly enhanced compared to traditional engineering features. Therefore, if all neighbors are selected for aggregation-as is done in traditional methods-it may lead to over-smoothing of node representations or the introduction of irrelevant information, which can negatively affect the model's performance and efficiency. To address this, we sample the neighbor nodes for aggregation using the aggregation strategy designed by the LLM.

Fig. 5 illustrates the performance of LTNA on the PUBMED and ARXIV datasets for different numbers of sampled neighbors $k \in \{2, 4, 6, 8, 10\}$. It can be observed that k follows different trends in the two datasets: in the PUBMED dataset, model performance increases initially and then decreases as k increases, while in the ARXIV dataset, model performance continues to improve as k increases.

To further investigate this phenomenon, we analyzed the relationship between this trend and the average node degree in each dataset. Since messages are propagated through neighboring nodes, the degree of the target node plays a crucial role. We calculated the average node degree to be 4.50 for PUBMED and 13.7 for ARXIV. The model performance improves when k is close to the average degree, as this satisfies the aggregated information needs for the majority of nodes. However, when k exceeds the average degree, it may lead to over-smoothing of node representations or including irrelevant information, thereby negatively impacting performance.

Considering the balance between computational resource constraints and model performance, we set k to 10 for experiments on the ARXIV dataset. Based on our analysis, it is expected that model performance could improve further if the number of samples is set close to the average degree (about 14).

6. Conclusion

In this paper, we explore the challenges of learning representations in TAGs and propose a novel approach LTNA. LTNA introduces a method for generating tailored aggregators for each node using LLMs to address the limitations of traditional GNNs, which typically rely on fixed aggregation strategies and struggle to capture the complexity of relationships between nodes. These limitations are particularly evident when significant differences exist between node features and neighboring information. By leveraging the semantic understanding and context-awareness of LLMs, LTNA dynamically adapts the aggregation strategy based on the unique context information of each node. Experimental results demonstrate that the proposed LTNA consistently outperforms other GNN models on five benchmark datasets. These results underscore the importance and necessity of adaptive aggregation in handling the complex relationships in TAGs and validate the effectiveness of using LLMs to enhance the aggregation process. Although LTNA effectively addresses the limitations of traditional fixed aggregation strategies, it still faces challenges in capturing global graph information and optimizing prompt design. First, the aggregation strategy considers only one-hop neighbors. In future work, we will explore multi-hop neighbor aggregation to further enhance the expressive power of node embeddings. Second, we plan to incorporate the reasoning capabilities of large models, enabling them to self-cue and optimize their own prompt design to enhance the aggregation effectiveness. Finally, while the introduction of LLM improves model performance, the computational cost remains high when handling large-scale datasets. Therefore, future research will focus on optimizing efficiency, which presents a highly promising direction.

CRedit authorship contribution statement

Songwei Zhao: Writing – original draft, Methodology, Conceptualization; **Yuan Jiang:** Writing – review & editing; **Sinuo Zhang:** Investigation; **Jifeng Hu:** Writing – review & editing; **Philip S. Yu:** Writing – review & editing; **Hechang Chen:** Writing – review & editing, Formal analysis, Conceptualization.

Declaration of competing interest

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

Acknowledgment

This work is supported in part by the [National Natural Science Foundation of China](#) (No. 62476110, No. U2341229); the NSF under grants III-2106758 and POSE-2346158; the Key R&D Project of Jilin Province (No. 20240304200SF).

Data availability

Data will be made available on request.

Appendix A. Notations

We conclude the commonly used notations in [Table A.1](#) for reference.

Table A.1
Major Hyperparameters and definitions.

Symbols	Definitions
$G = (\mathcal{V}, A, \{s_i\}_{i \in \mathcal{V}})$	the text-attributed graph
$\mathcal{V} = \{v_1, v_2, \dots, v_N\}$	the set of nodes in the graph
A	the adjacency matrix in the graph
N	the number of nodes in the graph
h_v^k	the embedding of node v at the k th layer
$\mathcal{N}(v)$	the neighbor set of node v
AGG	the aggregate function of GNN
UPD	the update function of GNN
$p(v)$	the prompt of node v
$\text{Select}_{\text{LLM}}$	the neighbor selection operation performed via LLM
$\text{Explain}_{\text{LLM}}$	the semantic parsing based on LLM
Act_{LLM}	the policy output operations for LLM
TAGG	the set of LLM-generated aggregation strategies
g_v	the tailored aggregation strategy (aggregator) for node v

Appendix B. Prompt template

To balance model safety and reproducibility, we have released LTNA's core prompt template as follows:

Prompt

Assume you are an experienced graph learning practitioner familiar with standard aggregators (max, sum, mean). Without revealing your reasoning chain, complete neighbor semantic determination and return an executable aggregation plan. Your task is to:

1. Based on the **Target Node Description** and **Neighbors Description**, first explain the semantic relations (supportive/critical/background/irrelevant) and select valuable neighbors.
2. Assign a semantic weight to each selected neighbor. Considering the distribution and interaction among selected neighbors, choose or design a text-aware nonlinear aggregator that best describes the integration mode between the target node and its neighbors (e.g., Expansive Referencing, Critical Referencing, Background-Driven Referencing). If needed, propose another suitable textual aggregation mode.
3. According to the designed aggregator, the information of the neighboring nodes is aggregated to the target node for updating its information.

We employ a design in which a single prompt encompasses the three-stage task, presenting the sub-tasks of select-explain-act within one unified instruction. The LLM sequentially completes these three logical components in a single forward inference. Specifically, the correspondence between each formula and the Prompt is as follows: Eq. (2) specifies the overall prompt template, i.e., the complete task description for the LLM. It includes three requirements: (i) selecting valuable neighbors, (ii) explaining semantic relationships, and (iii) designing aggregation patterns and outputting aggregation strategies. Eq. (3) corresponds to the first part of the first requirement in the prompt. Given the Target Node Description and Neighbors Description, the LLM selects valuable neighbors from the candidate set based on semantic information, thereby implementing the **Select** step. Eq. (4) provides another instruction corresponding to the first requirement in the prompt. It asks the LLM to explain the semantic relationship between each neighbor and the target node, thereby implementing the **Explain** step. Eq. (5) corresponds to the second requirement in the prompt. Based on the above analyses, the LLM is instructed to design a text-based aggregation pattern for the target node that best characterizes how information from its neighbors is integrated, thereby implementing the **Act** step.

Appendix C. Profile example for reasoning

To visually illustrate the model's operational mechanism, we present an idealized schematic diagram. This includes the interpretation of neighboring semantic relationships by $\text{Explain}_{\text{LLM}}$ and the numerical aggregation weights and nonlinear policies produced by Act_{LLM} . It is important to emphasize that this example is provided solely for explanatory purposes. In actual experiments, the model performs aggregation at the token level and does not produce explicit textual output.

Task Prompt

The **Prompt** content mentioned above. **Target Node Description:** For a newly released movie, I would like to go through everyone's post-viewing reviews as to whether or *Not* I would recommend going to see it and give advice on how much I can prescribe for fan merchandise to prevent being scammed.

Neighbors Description: Node 1: I found the movie absolutely captivating, with an engaging storyline and stellar performances that kept me enthralled. I was so impressed that I spent \$2,000 on fan merchandise to support my favorite actor. Node 2: I found the movie fairly entertaining, with a few notable moments, though it didn't leave a lasting impression. However, I liked the lead actor enough to spend \$500 on fan merchandise. Node 3: I didn't enjoy the movie, finding it boring and poorly made, but because I am such a devoted fan of the star, I still spent \$300 on fan merchandise to show my support.

Answers from the LLM

To help target node make an informed decision about watching the movie and assess potential fraud in purchasing the fan merchandise, we can design an aggregator that not only considers sentiment and spending but also provides a suitable price range for the product. This aggregator, called the Sentiment-Spending Fraud Detection Aggregator, works as follows: Steps of the Aggregator: Sentiment Analysis: First, we analyze the sentiment of each neighbor node's review to determine if their feedback is positive, neutral, or negative. This helps measure their emotional attachment to the movie and the product. We can assign a sentiment score ranging from -1 to 1, representing a scale from negative to positive sentiment. Node 1: Positive review, sentiment score of 0.9. Node 2: Neutral review, sentiment score of 0.3. Node 3: Negative review, sentiment score of -0.7. Spending Behavior Analysis: We evaluate how much each neighbor node spent on the same fan merchandise. Node 1 spent \$2,000. Node 2 spent \$500. Node 3 spent \$300. Bias Detection: A bias factor is introduced to measure the discrepancy between sentiment and spending. The greater the difference, the more likely the node is emotionally driven, increasing the risk of irrational spending. We calculate the sentiment-to-spending ratio to detect this bias: Node 1 has a positive sentiment but an extremely high spending, so the bias factor is: $\$2,000 / 0.9 = 2222.2\$$. Node 2 has moderate spending with neutral sentiment, resulting in a bias factor of: $\$500 / 0.3 = 1666.7\$$. Node 3 has negative sentiment but still spent \$300, indicating irrational fan-driven behavior. The bias factor is: $\$300 / -0.7 = -428.6\$$ (a negative value shows irrational negative spending). Fraud Detection and Price Suggestion: By comparing the bias factors, we can detect potential fraud. If a node's bias factor is significantly higher than others, it indicates that the product's price may be inflated. To determine an appropriate price range, we use the lowest bias factor as a reference and consider its spending behavior:

Continue

Node3/s negative bias factor suggests irrational spending, so the product's lower price limit should be below \$300. Node2/s bias factor is reasonable, so the product's upper price limit should not exceed \$500.

Weighted Aggregation: We combine the sentiment and bias factors from each node using a weighted average to make a balanced recommendation. Node 3 serves as the low-reference point, Node 2 as the middle reference, and Node 1 as the high-spending reference:

Node 1 is given a low weight of 0.2.

Node 2 is given a moderate weight of 0.5.

Node 3 is given a higher weight of 0.3.

Aggregate Information: Multiply each node's sentiment and bias factor by its weight, then compute the weighted average:

Node1/s sentiment contribution: $0.9 * 0.2 = 0.18$, spending bias: $2222.2 * 0.2 = 444.44$.

Node2/s sentiment contribution: $0.3 * 0.5 = 0.15$, spending bias: $1666.7 * 0.5 = 833.35$.

Node3/s sentiment contribution: $-0.7 * 0.3 = -0.21$, spending bias: $-428.6 * 0.3 = -128.58$.

The final aggregated sentiment score is:

TotalSentimentScore = $0.18 + 0.15 - 0.21 = 0.12$

The final total spending bias is: TotalBias = $444.44 + 833.35 - 128.58 = 1149.21$

Final recommendation: The overall sentiment score of 0.12 suggests a neutral perception of the movie, while the total spending bias of \$1,149.21 indicates that the product may be overpriced, suggesting potential fraud. Based on the behavior of Node 2 and Node 3, the reasonable price range for the product should be \$300 to \$500.

Aggregated information: "While the neighbors' overall sentiment toward the movie is neutral, their high spending on fan merchandise suggests emotionally driven behavior and potential product price inflation. A recommended price range for the product is \$300 to \$500 to avoid potential fraud."

References

- [1] K. Yao, J. Liang, J. Liang, M. Li, F. Cao, Multi-view graph convolutional networks with attention mechanism, *Artif. Intell.* 307 (2022) 103708.
- [2] G. Michel, G. Nikolentzos, J.F. Lutzeyer, M. Vazirgiannis, Path neural networks: expressive and accurate graph neural networks, in: *International Conference on Machine Learning*, PMLR, 2023, pp. 24737–24755.
- [3] S. Zhao, Y. Jiang, Z. Zhang, Y. Yu, H. Chen, GRAIN: multi-granular and implicit information aggregation graph neural network for heterophilous graphs, in: *Proceedings of the AAAI Conference on Artificial Intelligence*, vol. 39, 2025, pp. 13383–13391.
- [4] T. He, Y. Liu, Y.-S. Ong, X. Wu, X. Luo, Polarized message-passing in graph neural networks, *Artif. Intell.* 331 (2024) 104129.
- [5] W. Pei, W. Xu, Z. Wu, W. Li, J. Wang, G. Lu, X. Wang, Saliency-aware regularized graph neural network, *Artif. Intell.* 328 (2024) 104078.
- [6] X. Huang, K. Han, Y. Yang, D. Bao, Q. Tao, Z. Chai, Q. Zhu, GNNs as adapters for LLMs on text-attributed graphs, in: *The Web Conference 2024*, 2024.
- [7] T. Mikolov, I. Sutskever, K. Chen, G.S. Corrado, J. Dean, Distributed representations of words and phrases and their compositionality, *Adv. Neural Inf. Process. Syst.* 26 (2013) 1–9.
- [8] Z.S. Harris, Distributional structure, *Word*, 10 (2-3) (1954) 146–162.
- [9] X. Qiu, T. Sun, Y. Xu, Y. Shao, N. Dai, X. Huang, Pre-trained models for natural language processing: a survey, *Sci. China Technol. Sci.* 63 (10) (2020) 1872–1897.
- [10] A. Miaschi, F. Dell'Orletta, Contextual and non-contextual word embeddings: an in-depth linguistic investigation, in: *Proceedings of the 5th Workshop on Representation Learning for NLP*, 2020, pp. 110–119.
- [11] A. Chowdhery, S. Narang, J. Devlin, M. Bosma, G. Mishra, A. Roberts, P. Barham, H.W. Chung, C. Sutton, S. Gehrmann, et al., PaLM: scaling language modeling with pathways, *J. Mach. Learn. Res.* 24 (240) (2023) 1–113.
- [12] H. Touvron, T. Lavril, G. Izacard, X. Martinet, M.-A. Lachaux, T. Lacroix, B. Rozière, N. Goyal, E. Hambro, F. Azhar, et al., LLaMA: open and efficient foundation language models, (2023). [arXiv preprint arXiv:2302.13971](https://arxiv.org/abs/2302.13971)

- [13] J. Wei, M. Bosma, V.Y. Zhao, K. Guu, A.W. Yu, B. Lester, N. Du, A.M. Dai, Q.V. Le, Finetuned language models are zero-shot learners, (2021). [arXiv preprint arXiv:2109.01652](#)
- [14] G. Corso, L. Cavalleri, D. Beaini, P. Liò, P. Veličković, Principal neighbourhood aggregation for graph nets, *Adv. Neural Inf. Process. Syst.* 33 (2020) 13260–13271.
- [15] L. Yao, C. Mao, Y. Luo, Graph convolutional networks for text classification, in: *Proceedings of the AAAI Conference on Artificial Intelligence*, vol. 33, 2019, pp. 7370–7377.
- [16] S. Zhao, B. Yu, K. Yang, S. Zhang, J. Hu, Y. Jiang, P.S. Yu, H. Chen, A flexible diffusion convolution for graph neural networks, *IEEE Trans. Knowl. Data Eng.* 37 (6) (2025) 3118–3131.
- [17] Z. Chen, H. Mao, H. Li, W. Jin, H. Wen, X. Wei, S. Wang, D. Yin, W. Fan, H. Liu, et al., Exploring the potential of large language models (llms) in learning on graphs, *ACM SIGKDD Explor. Newsl.* 25 (2) (2024) 42–61.
- [18] Y. Fang, D. Fan, D. Zha, Q. Tan, GAUGLLM: improving graph contrastive learning for text-attributed graphs with large language models, in: *Proceedings of the 30th ACM SIGKDD Conference on Knowledge Discovery and Data Mining*, 2024, pp. 747–758.
- [19] A. Vaswani, Attention is all you need, *Adv. Neural Inf. Process. Syst.* 30 (2017) 1–11.
- [20] Z. Liu, X. Yu, Y. Fang, X. Zhang, GraphPrompt: unifying pre-training and downstream tasks for graph neural networks, in: *Proceedings of the ACM Web Conference 2023*, 2023, pp. 417–428.
- [21] T.N. Kipf, M. Welling, Semi-supervised classification with graph convolutional networks, (2016). [arXiv preprint arXiv:1609.02907](#)
- [22] P. Veličković, G. Cucurull, A. Casanova, A. Romero, P. Lio, Y. Bengio, Graph attention networks, (2017). [arXiv preprint arXiv:1710.10903](#)
- [23] K. Duan, Q. Liu, T.-S. Chua, S. Yan, W.T. Ooi, Q. Xie, J. He, SimTeG: a frustratingly simple approach improves textual graph learning, (2023). [arXiv preprint arXiv:2308.02565](#)
- [24] J. Tang, Y. Yang, W. Wei, L. Shi, L. Su, S. Cheng, D. Yin, C. Huang, GraphGPT: graph instruction tuning for large language models, in: *Proceedings of the 47th International ACM SIGIR Conference on Research and Development in Information Retrieval*, 2024, pp. 491–500.
- [25] T.B. Brown, Language models are few-shot learners, (2020). [arXiv preprint arXiv:2005.14165](#)
- [26] Z. Hou, H. Li, Y. Cen, J. Tang, Y. Dong, GraphAlign: pretraining one graph neural network on multiple graphs via feature alignment, (2024). [arXiv preprint arXiv:2406.02953](#)
- [27] C. Mavromatis, V.N. Ioannidis, S. Wang, D. Zheng, S. Adeshina, J. Ma, H. Zhao, C. Faloutsos, G. Karypis, Train your own GNN teacher: graph-aware distillation on textual graphs, in: *Joint European Conference on Machine Learning and Knowledge Discovery in Databases*, Springer, 2023, pp. 157–173.
- [28] J. Guo, L. Du, H. Liu, M. Zhou, X. He, S. Han, GPT4Graph: can large language models understand graph structured data? an empirical evaluation and benchmarking, (2023). [arXiv preprint arXiv:2305.15066](#)
- [29] W. Hamilton, Z. Ying, J. Leskovec, Inductive representation learning on large graphs, *Adv. Neural Inf. Process. Syst.* 30 (2017) 1–11.
- [30] C.-Y. Lee, P.W. Gallagher, Z. Tu, Generalizing pooling functions in convolutional neural networks: mixed, gated, and tree, in: *Artificial Intelligence and Statistics*, PMLR, 2016, pp. 464–472.
- [31] C. Cai, Y. Wang, A simple yet effective baseline for non-attributed graph classification, (2018). [arXiv preprint arXiv:1811.03508](#)
- [32] N. Dehmamy, A.-L. Barabási, R. Yu, Understanding the representation power of graph neural networks in learning graph topology, *Adv. Neural Inf. Process. Syst.* 32 (2019) 1–11.
- [33] H. Pei, Y. Li, H. Deng, J. Hai, P. Wang, J. Ma, J. Tao, Y. Xiong, X. Guan, Multi-track message passing: tackling oversmoothing and oversquashing in graph learning via preventing heterophily mixing, in: *Forty-first International Conference on Machine Learning*.
- [34] J.J. Rotman, *An Introduction to Algebraic Topology*, vol. 119, Springer Science & Business Media, 2013.
- [35] K. Borsuk, Drei Sätze über die n -dimensionale euklidische Sphäre, *Fundam. Math.* 20 (1) (1933) 177–190.
- [36] A.K. McCallum, K. Nigam, J. Rennie, K. Seymore, Automating the construction of internet portals with machine learning, *Inf. Retr.* 3 (2000) 127–163.
- [37] C.L. Giles, K.D. Bollacker, S. Lawrence, CiteSeer: an automatic citation indexing system, in: *Proceedings of the Third ACM Conference on Digital Libraries*, 1998, pp. 89–98.
- [38] S. Prithviraj, G. Namata, M. Bilgic, L. Getoor, B. Gallagher, T. Eliassi-Rad, Collective classification in network data, *AI Mag.* 29 (3) (2008) 93–93.
- [39] W. Hu, M. Fey, M. Zitnik, Y. Dong, H. Ren, B. Liu, M. Catasta, J. Leskovec, Open graph benchmark: datasets for machine learning on graphs, *Adv. Neural Inf. Process. Syst.* 33 (2020) 22118–22133.
- [40] X. He, X. Bresson, T. Laurent, A. Perold, Y. LeCun, B. Hooi, Harnessing explanations: LLM-to-LM interpreter for enhanced text-attributed graph representation learning, (2023). [arXiv preprint arXiv:2305.19523](#)
- [41] S. Abu-El-Haija, B. Perozzi, A. Kapoor, N. Alipourfard, K. Lerman, H. Harutyunyan, G. Ver Steeg, A. Galstyan, MixHop: higher-order graph convolutional architectures via sparsified neighborhood mixing, in: *International Conference on Machine Learning*, PMLR, 2019, pp. 21–29.
- [42] J. Gasteiger, A. Bojchevski, S. Günnemann, Predict then propagate: graph neural networks meet personalized pagerank, (2018). [arXiv preprint arXiv:1810.05997](#)
- [43] E. Chien, J. Peng, P. Li, O. Milenkovic, Adaptive universal generalized pagerank graph neural network, (2020). [arXiv preprint arXiv:2006.07988](#)
- [44] Y. Zhu, L. Feng, Z. Deng, Y. Chen, R. Amor, M. Witbrock, Robust node classification on graph data with graph and label noise, in: *Proceedings of the AAAI Conference on Artificial Intelligence*, vol. 38, 2024, pp. 17220–17227.
- [45] M. Park, J. Heo, D. Kim, Mitigating oversmoothing through reverse process of GNNs for heterophilic graphs, in: *Forty-first International Conference on Machine Learning*.
- [46] Y. Chen, L. Wu, M. Zaki, Iterative deep graph learning for graph neural networks: better and robust node embeddings, *Adv. Neural Inf. Process. Syst.* 33 (2020) 19314–19326.
- [47] Y. Gao, H. Yang, P. Zhang, C. Zhou, Y. Hu, GraphNAS: graph neural architecture search with reinforcement learning, (2019). [arXiv preprint arXiv:1904.09981](#)
- [48] J. Zhao, M. Qu, C. Li, H. Yan, Q. Liu, R. Li, X. Xie, J. Tang, Learning on large-scale text-attributed graphs via variational inference, (2022). [arXiv preprint arXiv:2210.14709](#)
- [49] Z. Guo, L. Xia, Y. Yu, Y. Wang, Z. Yang, W. Wei, L. Pang, T.-S. Chua, C. Huang, GraphEdit: large language models for graph structure learning, (2024). [arXiv preprint arXiv:2402.15183](#)
- [50] H. Liu, J. Feng, L. Kong, N. Liang, D. Tao, Y. Chen, M. Zhang, One for all: towards training one graph model for all classification tasks, (2023). [arXiv preprint arXiv:2310.00149](#)
- [51] N. Reimers, Sentence-BERT: sentence embeddings using siamese BERT-Networks, (2019). [arXiv preprint arXiv:1908.10084](#)
- [52] I. Beltagy, K. Lo, A. Cohan, SciBERT: a pretrained language model for scientific text, (2019). [arXiv preprint arXiv:1903.10676](#)
- [53] R. Anil, A.M. Dai, O. Firat, M. Johnson, D. Lepikhin, A. Passos, S. Shakeri, E. Taropa, P. Bailey, Z. Chen, et al., PaLM 2 technical report, (2023). [arXiv preprint arXiv:2305.10403](#)
- [54] J. Bai, S. Bai, Y. Chu, Z. Cui, K. Dang, X. Deng, Y. Fan, W. Ge, Y. Han, F. Huang, et al., Qwen technical report, (2023). [arXiv preprint arXiv:2309.16609](#)